DAT 471 – Computational Techniques for Large-Scale Data

Module 2

Group 6

Name	Programme	Email
Gabriela Castillo Avila	MPHPC	avilag@student.chalmers.se
Mahnoor Ali Cheema	MPDSC	mahnoora@student.chalmers.se

27 April 2026

Problem 1

Part a

It can be observed that speedup grows with n, but it is capped by the 30% sequential part. Using Amdahl's law:

$$S(n) = \frac{1}{(1-p) + \frac{p}{n}}$$

where:

- p = 0.70 (parallelizable fraction)
- 1 - p = 0.30 (sequential fraction)

Expected Speedups

For n = 2

$$S(2) = \frac{1}{0.3 + \frac{0.7}{2}} = \frac{1}{0.3 + 0.35} = \frac{1}{0.65} \approx 1.538$$

For n = 4

$$S(4) = \frac{1}{0.3 + \frac{0.7}{4}} = \frac{1}{0.3 + 0.175} = \frac{1}{0.475} \approx 2.105$$

For n = 8

$$S(8) = \frac{1}{0.3 + \frac{0.7}{8}} = \frac{1}{0.3 + 0.0875} = \frac{1}{0.3875} \approx 2.581$$

For n =16

$$S(16) = \frac{1}{0.3 + \frac{0.7}{16}} = \frac{1}{0.3 + 0.04375} = \frac{1}{0.34375} \approx 2.909$$

For n = 32

$$S(32) = \frac{1}{0.3 + \frac{0.7}{32}} = \frac{1}{0.3 + 0.021875} = \frac{1}{0.321875} \approx 3.106$$

Part b

The maximum theoretical speedup comes directly from Amdahl's law by letting the number of processors $n \to \infty$, and when n becomes infinitely large, the parallel part takes essentially zero time and thus only sequential part remains.
Sequential fraction = 1 - p = 0.30

Computing Speedup

$$S(\infty) = \frac{1}{1 - p + \frac{p}{\infty}} = \frac{1}{0.3 + 0} = \frac{1}{0.3} = 3.333$$

or we can say that, maximum theoretical speedup is:

$$S_{max} = 3.33$$

Which shows that even with unlimited processors, the pipeline cannot run more than about **3.33x faster** because 30% sequential part becomes the bottleneck.

Part c

In real world setting, the observed speedup will be **lower** than the theoretical values predicted by Amdahl's law. This is because real world system presents additional overheads that the idealized system does not account for, such as:

- Communication and synchronization delays between parallel tasks.
- Memory and I/O bottlenecks, such as contention for shared memory or disk access.
- Load imbalance, where processors do not receive equal amounts of work.
- Operating system and hardware noise, including context switching and interrupts.
- Parallelization overhead, such as thread creation and result aggregation.

These factors effectively increase the sequential portion of the workload and reduce the efficiency of the parallelizable part. As a result, real-world speedups typically falls below the theoretical curve and saturate earlier.
For this pipeline, a realistic maximum speedup would often lie in the range of 2.5–3.0×, depending on hardware and workload characteristics.

Problem 2

Part a

Part a consisted of implementing the functions *checksum* and *get_top10*. The checksum function was computed using the following mathematical formula in figure 1.

$$\sum_{i=1}^{n} |w_i| \cdot c_i \,,$$

Figure 1: Mathematical definition of checksum

The checksum was developed into the following simple function:

```
def compute_checksum(counts):
        checksum = 0
    for word, count in counts.items():
        checksum += len(word) * count
    return checksum
```

for the top_10 function the items were sorted in descending order and retrieving the top 10 candidates returning a list of (count, word) pairs:

```
def get_top10(counts):
    top10 = sorted(counts.items(), key=lambda x: x[1], reverse=True)[:10]
    top10 = [(count, word) for (word, count) in top10]
    return top10
```

The result for the checksum using the tiny dataset was 1885973, which verifies the correctness of the function. The result of the top 10 values with highest word count:

```
Top 10: [(20769, 'the'), (11549, 'of'), (10847, 'and'), (10107, 'to'),
(7737, 'a'), (6555, 'in'), (4690, 'that'), (4287, 'I'), (3866, 'his'),
(3748, 'was')]
```

Part b

The major parts of the program can be identified in the main function which consist of getting the files by traversing the directory tree and reading each file, counting the words of each respective file, merging per file into global_counts into one global dictionary.
The blocks within the main function consist of parsing the input, getting all the files, counting words, and merging.
The block that can be parallelized is the word counting as each file can be traversed independently.
The block that cannot be simply parallelized is the block in which the merge occurs, as each partial directory needs to be combined into a shared result, requiring workers to coordinate.

Part c

The big dataset was used in this problem and will be used continuously in the following parts for consistency, as the huge one resulted in a long runtime, with some errors.
The times of the different blocks and the total time are the following

```
Time to read files: 3.77 seconds
Time to count words: 44.03 seconds
Time to merge counts: 21.96 seconds
Total time: 69.77 seconds
```

The fractions spent in each individual block are the following

```
Fraction of time spent reading: 5.41%
Fraction of time spent counting: 63.11%
Fraction of time spent merging: 31.48%
```

since the block that was decided to be easily parallelized was the one that traverses files and counts the words, the theoretical speedup using Ahmdahl's law results in:

```
theoretical_speedup (Ahmdahls): 2.71
```

Part d

To parallelize the block that traverses the files and counts the words the code was changed into the following lines:

```
with mp.Pool(processes=num_workers) as pool:
file_counts = pool.map(count_words_in_file, files)
```

Number of workers	Runtime	Speedup (Tc/Tx)
1	144.24 seconds	0.48
2	65.93 seconds	1.06
4	47.54 seconds	1.47
8	42.24 seconds	1.65
16	39.78 seconds	1.75
32	41.04 seconds	1.70
64	43.52 seconds	1.60

Table 1: Runtime vs workers

The following code was used to run the program with different values of workers, and saved in a .txt file.

```
for w in 1 2 4 8 16 32 64;
do python assignment2_problem2_skeleton.py -w $w /data/.../gutenberg/big
>> results.txt;
done
```

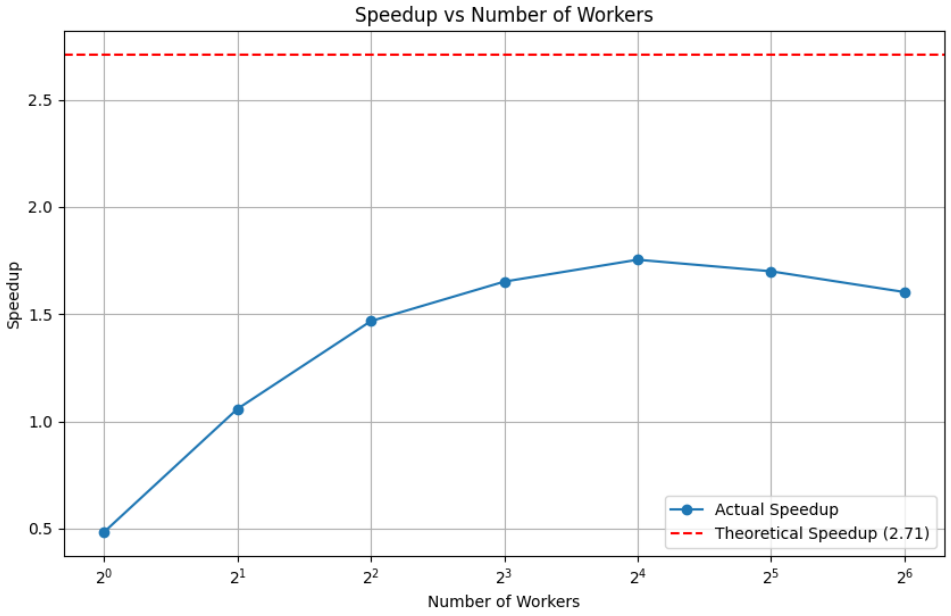


Figure 2: Plot of speedup as function of workers

in figure 2 the values calculated in table 1 are plotted as speedup as a function of workers along the calculated speedup, the dashed line represents the calculated speedup from part c of the assignment. The difference between the theoretical speedup from part c and the measured speedup in part d is due to the introduced multiprocessing overhead added to the runtime of the program. One worker doesn't benefit from the parallelization, therefore runs as serial with the added cost of the overhead in runtime, resulting in the increased time.

Part e

The adjustments for the program in the function *count_words_in_file* were the following:

```
def count_words_in_file(filename):

    file = get_file(filename)
    counts = dict()
    for word in file.split():
        if word in counts:
            counts[word] += 1
        else:
            counts[word] = 1
    return counts
```

And for the parallelized block in the main function:

```
filesname = list(get_filenames(path))
with mp.Pool(processes=num_workers) as pool:
    file_counts = pool.map(count_words_in_file, filesname)
```

Number of workers	Runtime	Speedup (Tc/Td)
1	84.08 seconds	0.83
2	54.98 seconds	1.27
4	39.33 seconds	1.77
8	31.93 seconds	2.19
16	30.05 seconds	2.32
32	29.77 seconds	2.34
64	29.93 seconds	2.4

Table 2: Runtime vs workers, file as input

Figure 3 represents the calculated speed up values from table 2. The increase in performance compared to figure 2, is due to the difference in where the work is performed and what data is sent to each worker. In the approach of part d, reading every file's content into memory is executed sequentially in the main process, and each worker is sent the entire file contents. In contrast, in this approach file reading is parallelized across workers and each worker receives filenames (small strings) instead of the full file content. This reduces the large amount of data transferred between processes and improves overall performance.

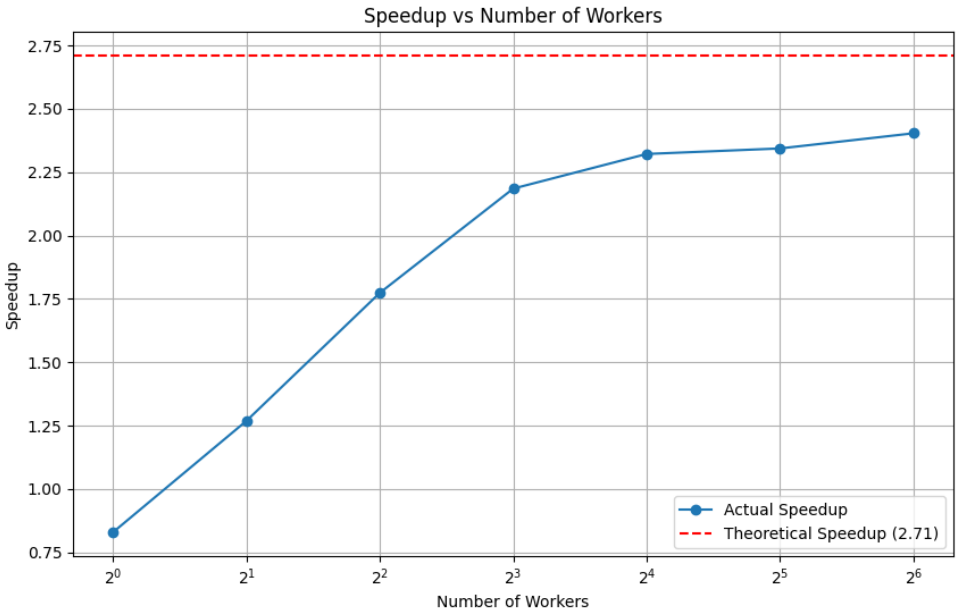


Figure 3: Plot of speedup as function of workers

Part f

for the implementation of this section the following helper functions were created: Word count using workers:

```
def count_words_in_file(filename_queue,wordcount_queue,batch_size):
      worker_word_counts = {}
  batch_size_counter = 0
  while True:
      file = filename_queue.get()
      if file is None:
          if worker_word_counts:
              wordcount_queue.put(worker_word_counts)
          wordcount_queue.put(None)
          break
      content_file = get_file(file)
      for word in content_file.split():
          if word in worker_word_counts:
              worker_word_counts[word] += 1
          else:
              worker_word_counts[word] = 1
      batch_size_counter += 1
      if batch_size_counter >= batch_size:
          wordcount_queue.put(worker_word_counts)
          worker_word_counts = {}
          batch_size_counter = 0
```

Merge using workers:

```
def merge_counts(out_queue,wordcount_queue,num_workers):
  global_counts = {}
  workers_done = 0
  while workers_done < num_workers:
      counts_to_merge = wordcount_queue.get()
      if counts_to_merge is None:
          workers_done += 1
      else:
          for (k,v) in counts_to_merge.items():
              if k not in global_counts:
                  global_counts[k] = v
              else:
                  global_counts[k] += v

  checksum = compute_checksum(global_counts)
  top10 = get_top10(global_counts)
  out_queue.put((top10, checksum))
```

to find a suitable batch size experiments were performed using 64 workers with a batch size of 10 20 ... 150. leading to the conclusion of the appropriate batch size being 90.

Number of workers	Runtime	Speedup (Tc/Tf)
1	67.75 seconds	1.03
2	40.13 seconds	1.74
4	25.49 seconds	2.74
8	25.35 seconds	2.75
16	25.54 seconds	2.73
32	26.12 seconds	2.67
64	27.52 seconds	2.54

Table 3: Runtime vs workers, using queues

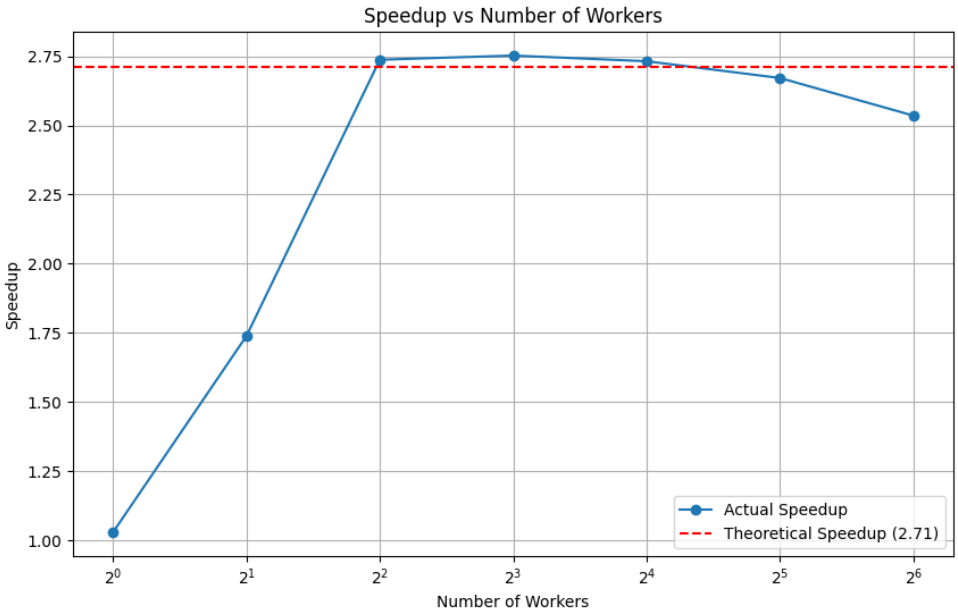


Figure 4: Plot of speedup as function of workers

Table 3 and figure 4 illustrate that the theoretical speedup calculated in part c is a conservative estimate. The model in part c assumes that only the word-counting block is parallelizeable. However, in part f, file reading is also performed by the workers and the merging block is partially overlapping with worker execution. As a result the speedup due to the parallel fraction exceeds the estimate in part c, which explains why the measured speedup in this part exceeds the theoretical bound.

Part g

Top 10 most common words using 64 workers on the huge dataset:

- the = 148499457
- of = 89408738
- and = 72120653
- to = 61762640
- a = 51211506
- in = 44714455
- that = 24646774
- was = 24224366
- is = 19939759
- i = 19103566